SMART STUDY RECALL / ENGINEERING AND RESEARCH GUIDE

Deep Knowledge-Tracing Models

Detailed implementation of the MLP, LSTM and causal Transformer candidates

Prepared from the repository implementation and experiment reports as inspected on 27 October 2026.

Faculty presentation edition | Implementation facts are separated from research hypotheses and unvalidated claims.

How to use this document

This guide explains the running implementation, what data enters each component, what algorithms do with it, what is saved, how to operate the project, and what the current experiments do and do not establish. Formulae use plain ASCII notation. Source paths at the end let a reviewer trace each description back to the repository.

Important interpretation rule: a predicted next-session correct fraction is a score proxy. It is not a measured probability that a learner will remember a course concept on a future date.

Contents

01. 1. What knowledge tracing means here

02. 2. One event token and the causal boundary

03. 3. Shared input embedding and output head

04. 4. MLP: order-agnostic feedforward baseline

05. 5. LSTM: gated sequential memory

06. 6. Transformer: causal self-attention

07. 7. Objective, optimizer and training

08. 8. Recorded results and interpretation

09. 9. Inference API and non-use in planner

10. 10. Main limits and next engineering steps

1. What knowledge tracing means here

The experimental knowledge-tracing endpoint estimates the next interaction's observed session_correct / session_seen fraction. It receives timestamp-ordered learner-item interactions, a target item ID, pre-event history counts, and elapsed time. It is not a delayed-retention probability and it is not the course-topic model currently used for planner dates.

Three neural candidates share one causal data contract: an MLP summary baseline, a two-layer LSTM and a causal Transformer encoder. All output a scalar through sigmoid, placing the model's estimate between 0 and 1.

Why keep this model separate? Duolingo lexeme IDs, response format and user population do not match the app's topic IDs and uploaded course concepts. Wiring the model into the forgetting-date dashboard would imply unsupported cross-domain validity.

2. One event token and the causal boundary

Input component	Meaning at prediction time
Item embedding, 32 dimensions	Known lexeme ID represented by a learned vector; index 0 is padding, index 1 is OOV
log_history_seen	ln(1 + prior responses for this learner and item)
history_accuracy	prior correct divided by prior seen; zero if no history
log_gap_days	ln(1 + elapsed days since previous learner timestamp group)
previous_session_accuracy	correct fraction in the previous timestamp group, not the target group

The item vocabulary and numeric scaler are fit using training data only. Unknown validation/test/inference target items use OOV index 1. Padding uses index 0 and is masked. An event window has length 30. Each target token contains target item identity plus information available before that target's outcome.

Rows sharing the same learner timestamp are grouped as one information boundary. A target cannot see another answer from its own simultaneous group. Only after the group is observed does its aggregated accuracy become prior_session_accuracy for a later timestamp. This prevents within-session answer leakage.

For forward-time evaluation, the event-count cutoffs snap to complete timestamp groups. The train/validation/test proportions are 70/15/15, with preprocessing fit on training rows only. Target fractions are weighted by session_seen in the loss/aggregate metrics, so an event representing more answers contributes more evidence.

3. Shared input embedding and output head

For each sequence position, PyTorch looks up the 32-dimensional item embedding and concatenates four standardized numeric features. A shared projection applies Linear(36,64), LayerNorm(64), then GELU. This produces a 64-dimensional event representation for the chosen architecture.

After sequence encoding, a head maps the selected representation through Linear(64,32), GELU, Dropout(0.2), Linear(32,1), then sigmoid. The MLP receives a wider 128-dimensional concatenation of current event representation and masked mean-history representation before its own hidden layers.

Dropout randomly zeros activations during training to discourage co-adaptation. It is disabled in evaluation mode. Layer normalization stabilizes the per-event projection. GELU is a smooth nonlinear activation used in the event projection and sequence-model heads.

4. MLP: order-agnostic feedforward baseline

The MLP is not a sequence-aware neural net even though it receives a window. It encodes each event, then computes the masked arithmetic mean of all valid event representations. It separately selects the final valid representation as the current target event. Concatenating these gives a compact current-plus-history summary.

```
valid_mask = 1 for real tokens, 0 for padding
mean_context = sum(encoded_t * valid_mask_t) / valid_token_count
current = encoded[last_valid_position]
summary = concatenate(current, mean_context)
score = sigmoid(MLP(summary))
```

The masked mean discards the ordering of earlier history; it exposes the average represented context but not whether one event came before another. The current event remains distinct. Its feedforward head is Linear(128,64), GELU, Dropout, Linear(64,32), GELU, Dropout, Linear(32,1), sigmoid.

Interpret the MLP as a nonrecurrent neural baseline. It tests whether item representations plus a pooled summary are sufficient compared with explicit temporal encoders.

---

5. LSTM: gated sequential memory

An LSTM processes valid tokens in order. Its cell state and hidden state use gates that regulate what information is retained, forgotten or exposed at each step. This is intended to model sequence effects over the preceding item interactions.

The implementation uses two stacked LSTM layers, hidden size 64, batch-first layout and dropout 0.2 between recurrent layers. Variable-length padded windows are packed using pack_padded_sequence, so padding positions do not act as real events. The output at the last valid token is passed through the shared scalar head and sigmoid.

```
i_t = sigmoid(W_i x_t + U_i h_(t-1) + b_i)
f_t = sigmoid(W_f x_t + U_f h_(t-1) + b_f)
o_t = sigmoid(W_o x_t + U_o h_(t-1) + b_o)
g_t = tanh(W_g x_t + U_g h_(t-1) + b_g)
c_t = f_t * c_(t-1) + i_t * g_t
h_t = o_t * tanh(c_t)
```

This LSTM is a small sequence model, not an unrestricted long-term memory mechanism: it receives no more than 29 preceding events plus the current target token in the sequence construction. The correct outcome for the current target is unavailable to its own input.

---

6. Transformer: causal self-attention

A Transformer encoder compares token representations using multi-head self-attention. Here each 64-dimensional event representation is augmented by a learned position embedding so sequence order is represented. The encoder has two layers, four attention heads, feed-forward width 128, dropout 0.2, batch-first layout and norm_first enabled.

A causal upper-triangular attention mask prevents position t from reading positions after t. A separate key-padding mask excludes padded tokens. The representation at the last valid position goes to the shared MLP prediction head.

```
Q = X W_Q; K = X W_K; V = X W_V
Attention(Q,K,V) = softmax((Q K^T) / sqrt(d_k) + causal_mask) V
MultiHead = concatenate(head_1, ..., head_4) W_O
Each block uses pre-layer normalization, attention and a 128-wide feed-forward sublayer with
residual connections.
```

Self-attention provides a direct path for one event representation to use relevant earlier positions without compressing all history through a single recurrent state. The causal mask and same-timestamp group boundary enforce direction of

information. The positional embedding is learned within the fixed 30-position window.

---

7. Objective, optimizer and training

The target is the aggregated correct fraction y = session_correct / session_seen. Binary cross entropy is evaluated on the fraction and weighted by session_seen, which is equivalent to treating that fraction as the summary of a binomial count. Batch loss is normalized by total seen count.

```
per_event_loss = BCE(predicted_fraction, observed_fraction)
weighted_loss = sum(session_seen * per_event_loss) / sum(session_seen)
```

Validation weighted binomial loss selects the checkpoint. Early stopping uses patience 2; the best validation checkpoint is restored. Training supports AdamW, Adam, RMSProp or momentum SGD; default runs use AdamW at learning rate 0.001 and weight decay 1e-4, while SGD uses learning rate 0.01 and momentum 0.9. Training can use CPU or GPU; recorded reports are CPU runs.

The forward-time neural comparison used seed 42 and a deterministic 1-in-100 sample. The MLP requested up to eight epochs and stopped after four, with best checkpoint epoch 2. LSTM and Transformer each used a three-epoch maximum. The held-out learner MLP/LSTM comparison uses a smaller 1-in-500 sample. These budgets are not a repeated, fully matched model-selection study.

---

8. Recorded results and interpretation

Forward-time model	Weighted MAE	Weighted RMSE	Binomial log loss	ECE	Weighted AUC
Mean	0.1525	0.2266	0.3241	0.0000	0.5000
Random Forest	0.1472	0.2225	0.3170	0.0104	0.5959
MLP AdamW	0.1444	0.2247	0.3200	0.0082	0.5869
LSTM	0.1468	0.2246	0.3195	0.0108	0.5927
Transformer	0.1573	0.2249	0.3203	0.0154	0.5851

On the 1-in-100 forward split, MLP has the lowest neural-model MAE. Random Forest has lower RMSE, log loss and higher AUC than the neural models. The mean baseline's extremely low ECE reflects near-constant predictions and does not mean useful discrimination. Transformer does not win this run.

Learner-held-out, 1-in-500	Weighted MAE	Weighted RMSE	Log loss	ECE	AUC
Mean	0.1330	0.2091	0.2666	0.0120	0.5000
Random Forest	0.1254	0.2061	0.2583	0.0085	0.6266
MLP AdamW	0.1338	0.2084	0.2648	0.0156	0.5795
LSTM	0.1442	0.2110	0.2707	0.0296	0.4883

On the smaller held-out-learner run, Random Forest beats the shown neural models on MAE and AUC. Transformer has no corresponding run in this table. This is one seed and no learner-bootstrap interval; report it as preliminary.

The one-seed MLP optimizer comparison finds Adam lowest on test MAE, RMSProp best on log loss/RMSE, and SGD lowest on ECE. No optimizer wins every measure. The reported budgets differ: the AdamW run early-stopped at epoch 2 with a later requested cap, while others used three epochs.

9. Inference API and non-use in planner

POST /predict/knowledge-tracing accepts up to 500 time-ordered interactions, a target_item_id, target_history_seen, target_history_correct, elapsed_days and architecture mlp, lstm or transformer. The API validates count consistency and nondecreasing timestamps. Serving loads the matching trained forward-time checkpoint, builds a causal target window, applies the saved scaler, executes the model without gradient calculation and returns the fraction, artifact hash, history count, cold-start flag, vocabulary-match flag, experimental status and calibrated=false.

This endpoint runs as a separate local ML API. It is not proxied into the app dashboard because course-topic IDs and Duolingo lexeme IDs are not aligned. The dashboard's course topic forecast uses the separate validation-selected next-session model, currently HLR. Do not say the LSTM is producing the dashboard forgetting date.

10. Main limits and next engineering steps

- Run more seeds and both split protocols on identical samples for every architecture. Add learner-cluster confidence intervals.
- Give Random Forest the same event history summary or present current results as non-feature-controlled comparisons.
- Evaluate probability calibration and choose thresholds on validation only. Report calibration curves and uncertainty on held-out data.
- Acquire task-matched, consented course interaction and delayed assessment data before connecting KT to course recall.

An LSTM or Transformer is not automatically more accurate than a Random Forest. Model family describes how it processes inputs; performance is an empirical result on a declared target, sample and split.

Repository traceability and source files

The report describes the local repository snapshot, not an independently deployed cloud instance. A running deployment can differ if its backend, database migrations, model artifacts or environment variables have not been updated.

Primary source files reviewed:

- docs/MODEL_ARCHITECTURE.md
- docs/DATASET_AND_PREPROCESSING.md
- docs/EXPERIMENT_PROTOCOL.md
- docs/RESULTS.md
- ml/src/knowledge_tracing.py
- ml/src/knowledge_tracing_service.py
- ml/src/train_knowledge_tracer.py
- ml/reports/knowledge_tracing_*_evaluation.json